FINAL Unit1-Unit2 FRQ - Topic order

This question involves the RaceCar and SimulateCarRace classes, which are used to simulate a race between two race cars. Each race consists of one or more laps around a racetrack, starting with lap 1. Each lap consists of one complete trip around the racetrack.

The RaceCar class contains a helper method, getLapTime. You will write two methods in the SimulateCarRace class.

```
public class RaceCar
{
  /**
   *  Returns the number of seconds it takes the race car
   *  to complete the lap specified by the parameter
   */
  public double getLapTime(int lapNum)
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

```
public class SimulateCarRace
{
  private RaceCar carOne;
  private RaceCar carTwo;

  /**
   *  Simulates a race with numLaps laps and returns a String
   *  indicating the outcome, as described in part (a)
   *  Precondition: numLaps > 0
   */
  public String findWinner(int numLaps)
  { /* to be implemented in part (a) */}

  /**
   *  Returns a shortened version of the parameter message with all
   *  substrings that are equal to the parameter str removed, as
   *  described in part (b)
   *  Precondition: There are no overlapping occurrences of str
   *                within message.
   */
  public String shortenMessage(String message, String str)
  { /* to be implemented in part (b) */}

  /* There may be instance variables, constructors, and methods
     that are not shown. */
}
```

1. Write the findWinner method, which simulates a race between two cars consisting of numLaps laps around a racetrack. Each car takes a different amount of time to complete each lap, which can be found using the getLapTime method in the

FINAL Unit1-Unit2 FRQ - Topic order

RaceCar class. For example, calling getLapTime(3) on a RaceCar object will return the number of seconds it takes the car to complete lap 3 of the race.

The findWinner method returns a String indicating the outcome of the race, according to the following rules.

- "Car 1 wins!" is returned if carOne completes the race in fewer seconds than carTwo.
- "Car 2 wins!" is returned if carTwo completes the race in fewer seconds than carOne.
- "Tie!" is returned if the cars complete the race in the same amount of time.

Complete method findWinner. You must use getLapTime appropriately in order to receive full credit.

```
/**
 * Simulates a race with numLaps laps and returns a String
 * indicating the outcome, as described in part (a)
 * Precondition: numLaps > 0
 */
public String findWinner(int numLaps)
```

2. Write the shortenMessage method, which returns a shortened version of the parameter message with all substrings that are equal to the parameter str removed. If str does not appear in message, the original value of message is returned.

For example, shortenMessage("the_driver_of_the_first_car_is_Jerry", "the_") should return "driver_of_first_car_is_Jerry".

Complete method shortenMessage.

```
/**
 * Returns a shortened version of the parameter message with all
 * substrings that are equal to the parameter str removed, as
 * described in part (b)
 * Precondition: There are no overlapping occurrences of str
 *               within message.
 */
public String shortenMessage(String message, String str)
```

This question involves the MessageBuilder class, which is used to generate a message based on a starting word. The MessageBuilder class contains a helper method, getNextWord. You will write a constructor and a method in the MessageBuilder class.

```
public class MessageBuilder
{
  private String message;   // To be initialized in part (a)
  private int numWords;     // To be initialized in part (a)

  /**
   * Builds a message starting with the word specified by the parameter and
   * counts the number of words in the message, as described in part (a)
```

FINAL Unit1-Unit2 FRQ - Topic order

```
 * Precondition: startingWord is a single word with no spaces.
 */
public MessageBuilder(String startingWord)
{ /* to be implemented in part (a) */}

/**
 * Returns a word to follow the word specified by the
 * parameter or null if there are no remaining words.
 * Precondition: s is a single word with no spaces.
 * Postcondition: Returns an individual word with no spaces.
 */
public String getNextWord(String s)
{ /* implementation not shown */ }

/**
 * Returns an abbreviation for the instance variable message,
 * as described in part (b)
 * Preconditions: Each word in message is separated by a single space.
 *              message contains two or more words.
 * Postcondition: message is unchanged.
 */
public String getAbbreviation()
{ /* to be implemented in part (b) */}

/* There may be instance variables, constructors, and methods
   that are not shown. */
}
```

3. Write the MessageBuilder constructor, which uses a helper method to create a message starting with the word specified by the parameter startingWord and assigns the message to the instance variable message. The constructor also counts the number of words in the message and assigns the count to the instance variable numWords.

Each word in the message will be separated by a single space.

A helper method, getNextWord, has been provided to obtain words to be added to the message. Each call to getNextWord returns the next word in the message based on the word most recently added to the message. When there are no more words to be added to the message, getNextWord returns null. Consecutive calls to getNextWord are guaranteed to eventually return null.

Example 1

Consider the following calls to getNextWord, which are made within the MessageBuilder class. The return value of each call is used in the next call.

Method Call	Return Value
getNextWord("the")	"book"
getNextWord("book")	"on"

FINAL Unit1-Unit2 FRQ - Topic order

Method Call	Return Value
getNextWord("on")	"the"
getNextWord("the")	"table"
getNextWord("table")	null

Based on these return values, a call to the MessageBuilder constructor with argument "the" should set the instance variable message to "the book on the table" and should set the instance variable numWords to 5.

Example 2

Consider the following calls to getNextWord, which are made within the MessageBuilder class. The return value of each call is used in the next call.

Method Call	Return Value
getNextWord("good")	"morning"
getNextWord("morning")	"sunshine"
getNextWord("sunshine")	null

Based on these return values, a call to the MessageBuilder constructor with argument "good" should set the instance variable message to "good morning sunshine" and should set the instance variable numWords to 3.

Complete the MessageBuilder constructor. You must use getNextWord appropriately in order to receive full credit.

```
/**
 * Builds a message starting with the word specified by the parameter and
 * counts the number of words in the message, as described in part (a)
 * Precondition: startingWord is a single word with no spaces.
 */
public MessageBuilder(String startingWord)
```

4. Write the getAbbreviation method, which returns an abbreviation consisting of the first letter of each word in message. Assume that message consists of two or more words, each separated by a single space.

For example, if the value of message is "as soon as possible", then getAbbreviation() should return "asap".

FINAL Unit1-Unit2 FRQ - Topic order

Complete the getAbbreviation method.

```
/**
 * Returns an abbreviation for the instance variable message,
 * as described in part (b)
 * Preconditions: Each word in message is separated by a single space.
 *                message contains two or more words.
 * Postcondition: message is unchanged.
 */
public String getAbbreviation()
```

This question involves writing two code segments that use information about downloaded songs.

A record company uses the following variables to maintain information about each song in the company's catalog.

- `downloads` is an `int` variable that represents the number of times a song was downloaded. Assume the value of this variable is positive.
- `songInfo` is a `String` variable that contains information about the song. The string consists of the artist name, followed by a hyphen ("-"), followed by the album title, followed by another hyphen, followed by the song title. Assume that no other hyphens appear in the string.

5. A code segment, which you will write, should determine whether the ones digit of the number stored in downloads appears exactly one other time in the number. If so, the code segment should print "exactly once"; otherwise, it should print "not exactly once".

For example, if downloads has a value of 1734567, then the segment should print "exactly once" because the ones digit, which is 7, appears one other time.

As another example, if downloads has a value of 325322, then the segment should print "not exactly once" because the ones digit, which is 2, appears two other times.

Write a code segment to print the intended output. Your implementation must conform to the examples.

6. A code segment, which you will write, should process the string songInfo and print the song title and artist name separated by the string (" by ").

For example, if songInfo is "Carrie Ferriero-Greatest Hits-Tell Me Why", then the code segment should print "Tell Me Why by Carrie Ferriero".

As another example, if songInfo is "Aniita-Mixtape 2-How Much More", then the code segment should print "How Much More by Aniita".

Write a code segment to print the intended output. Your implementation must conform to the examples.

This question involves processing strings of text.

FINAL Unit1-Unit2 FRQ - Topic order

Consider the following variables.

- colWidth is an int variable with a positive value.
- text is a String variable with a non-null value.
- otherText is a String variable with a non-null value.

7. A code segment, which you will write, should print output according to the following rules.

- If the length of text is greater than colWidth, then the value of text will be printed.
- Otherwise, the printed output will consist of colWidth characters contained between two "|" characters. The text between the "|" characters will consist of a sequence of periods ("."), followed by the value of text, followed by another sequence of periods.

  The printed output will have an equal number of periods on each side of text, if possible. If this is not possible, then there will be one fewer period on the left side of text than on the right side of text.

The intended behavior is illustrated in the following table.

Value of colWidth	Value of text	Printed Output
5	**"ab"**	|.ab..|
5	**"abc"**	|.abc.|
10	**"ab"**	|....ab....|
10	**"abc"**	|...abc....|
10	**"abcdefghij"**	|abcdefghij|
10	**"abcdefghijklmnopqrstuv"**	abcdefghijklmnopqrstuv

Write a code segment to print the intended output. Your implementation must meet all specifications and conform to the examples shown in the table.

8. A code segment, which you will write, should print the length of the longest prefix of the text that is a substring of otherText. A prefix is a substring that occurs at the beginning of another string.

The intended behavior is illustrated in the following table.

Value of text	Value of otherText	Printed Output	Explanation
"apple"	**"pear"**	1	The prefix "a" is a substring of "pear", but "ap" is not.

FINAL Unit1-Unit2 FRQ - Topic order

Value of text	Value of otherText	Printed Output	Explanation
"apple"	"pineapple"	5	The prefixes "a", "ap", "app", "appl", and "apple" are substrings of "pineapple". The length of the longest substring is 5.
"apple"	"happy"	3	The prefixes "a", "ap", and "app" are substrings of "happy". The length of the longest substring is 3.
"apple"	"leek"	0	No prefix of "apple" is a substring of "leek".

Write a code segment to print the intended output. Your implementation must meet all specifications and conform to the examples shown in the table.

This question involves the RobotMover class, which maintains a string that represents the moves made by a robot. You will write one constructor and one method in the RobotMover class.

```
public class RobotMover
{
  private String moveSequence;

  /**
   *  Initalizes moveSequence with numMoves moves, with an equal
   *  chance of "up", "down", "left", and "right". Each move is
   *  followed by an underscore ("_").
   *  Precondition: numMoves > 0
   */
  public RobotMover(int numMoves)
  { /* to be implemented in part (a) */ }

  /**
   *  Returns the number of times that str appears in moveSequence
   *  Precondition:  moveSequence is a valid sequence of moves,
   *                 each followed by an underscore.
   *  Postcondition: moveSequence is unchanged.
   */
  public int countOccurrences(String str)
  { /* to be implemented in part (b) */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

9. Write the RobotMover constructor, which initializes the instance variable moveSequence with numMoves moves, with each move followed by an underscore ("_"). Each move should be randomly selected, with an equal chance of being "up", "down", "left", or "right".

For example, new RobotMover(6) will initialize moveSequence with six moves. Some possible values of moveSequence after the call are "up_right_right_left_down_up_" or "up_up_down_down_left_right_".

Complete the RobotMover constructor.

```
/**
 * Initalizes moveSequence with numMoves moves, with an equal
 * chance of "up", "down", "left", and "right". Each move is
 * followed by an underscore ("_").
 * Precondition: numMoves > 0
 */
public RobotMover(int numMoves)
```

10. Write the countOccurrences method, which returns the number of times that the parameter str appears in moveSequence.

For example, suppose moveSequence is "right_down_left_down_right_down_up_up_up_". For this example, countOccurrences should behave as follows.

- The call countOccurrences("right") should return 2.
- The call countOccurrences("down") should return 3.
- The call countOccurrences("right_d") should return 2.
- The call countOccurrences("up_up") should return 2.

Complete method countOccurrences.

```
/**
 * Returns the number of times that str appears in moveSequence
 * Precondition:  moveSequence is a valid sequence of moves,
 *                each followed by an underscore.
 * Postcondition: moveSequence is unchanged.
 */
public int countOccurrences(String str)
```

---

11. **SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.**

Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question simulates birds or possibly a bear eating at a bird feeder. The following `Feeder` class contains information about how much food is in the bird feeder and simulates how much food is eaten. You will write two methods of the `Feeder` class.

```
public class Feeder
{
  /**
```

FINAL Unit1-Unit2 FRQ - Topic order

```
 * The amount of food, in grams, currently in the bird feeder; initialized in
the constructor and
 * always greater than or equal to zero
 */
private int currentFood;
/**
 * Simulates one day with numBirds birds or possibly a bear at the bird
feeder,
 * as described in part (a)
 * Precondition: numBirds > 0
 */
public void simulateOneDay(int numBirds)
{ /* to be implemented in part (a) */ }
/**
 * Returns the number of days birds or a bear found food to eat at the feeder
in this simulation,
 * as described in part (b)
 * Preconditions: numBirds > 0, numDays > 0
 */
public int simulateManyDays(int numBirds, int numDays)
{ /* to be implemented in part (b) */ }
// There may be instance variables, constructors, or methods that are not
shown.
}
```

(a) Write the `simulateOneDay` method, which simulates `numBirds` birds or possibly a bear at the feeder for one day. The method determines the amount of food taken from the feeder on this day and updates the `currentFood` instance variable. The simulation accounts for normal conditions, which occur 95% of the time, and abnormal conditions, which occur 5% of the time.

Under normal conditions, the simulation assumes that on any given day, only birds visit the feeder and that each bird at the feeder consumes the same amount of food. This standard amount consumed is between 10 and 50 grams of food, inclusive, in 1-gram increments. That is, on any given day, each bird might eat 10, 11, . . . , 49, or 50 grams of food. The amount of food eaten by each bird on a given day is randomly generated and each integer from 10 to 50, inclusive, has an equal chance of being chosen.

For example, a run of the simulation might predict that for a certain day under normal conditions, each bird coming to the feeder will eat 11 grams of food. If 10 birds come to the feeder on that day, then a total of 110 grams of food will be consumed.

If the simulated food consumed is greater than the amount of food in the feeder, the birds empty the feeder and the amount of food in the feeder at the end of the day is zero.

Under abnormal conditions, a bear empties the feeder and the amount of food in the feeder at the end of the day is zero.

The following examples show possible results of three calls to `simulateOneDay`.

Example 1: If the feeder initially contains 500 grams of food, the call `simulateOneDay(12)` could result in 12 birds eating 20 grams of food each, leaving 260 grams of food in the feeder.

Example 2: If the feeder initially contains 1,000 grams of food, the call `simulateOneDay(22)` could result in a bear eating all the food, leaving 0 grams of food in the feeder.

Example 3: If the feeder initially contains 100 grams of food, the call `simulateOneDay(5)` could result in 5 birds attempting to eat 30 grams of food each. Since the feeder initially contains less than 150 grams of food, the feeder is emptied, leaving 0 grams of food in the feeder.

Complete the `simulateOneDay` method.

```
/**
* Simulates one day with numBirds birds or possibly a bear at the bird feeder,
* as described in part (a)
* Precondition: numBirds > 0
*/
public void simulateOneDay(int numBirds)
```

Class information for this question

```
public class Feeder
private int currentFood
public void simulateOneDay(int numBirds)
public int simulateManyDays(int numBirds, int numDays)
```

(b) Write the `simulateManyDays` method. The method uses `simulateOneDay` to simulate `numBirds` birds or a bear coming to the feeder on at most `numDays` consecutive days. The simulation returns the number of days that birds or a bear found food at the feeder.

Consider the following examples.

Value of `currentFood` and Method Call	Possible Outcomes and Resulting Return Value
`currentFood: 2400` `simulateManyDays(10, 4)`	Day 1: `simulateOneDay` leaves 2100 grams of food in the feeder. Day 2: `simulateOneDay` leaves 1650 grams of food in the feeder. Day 3: `simulateOneDay` leaves 1500 grams of food in the feeder. Day 4: `simulateOneDay` leaves 1260 grams of food in the feeder. The simulation returns `4` because, on all four days of the simulation, birds or a bear found food at the feeder. The instance variable `currentFood` has the value `1260`.
`currentFood: 250` `simulateManyDays(10, 5)`	Day 1: `simulateOneDay` leaves 150 grams of food in the feeder. Day 2: `simulateOneDay` leaves 0 grams of food in the feeder. The simulation returns `2` because, on two of the five simulated days, birds or a bear found food at the feeder. The instance variable `currentFood` has the value `0`.
`currentFood: 0` `simulateManyDays(5, 10)`	The simulation returns `0` because no food was found at the feeder on any day. The instance variable `currentFood` has the value `0`.

Complete the `simulateManyDays` method. Assume that `simulateOneDay` works as intended, regardless of what you wrote in part (a). You must use `simulateOneDay` appropriately in order to receive full credit.

```
/**
* Returns the number of days birds or a bear found food to eat at the feeder
in this simulation,
* as described in part (b)
* Preconditions: numBirds > 0, numDays > 0
*/
public int simulateManyDays(int numBirds, int numDays)
```

Class information for this question

FINAL Unit1-Unit2 FRQ - Topic order

```
public class Feeder
private int currentFood
public void simulateOneDay(int numBirds)
public int simulateManyDays(int numBirds, int numDays)
```

This question involves processing a String that represents a sequence of beads.

The String variable beadSequence represents a bead sequence. It contains a sequence of letters in which each letter represents the color of a bead.

A bead sequence is considered acceptable if it adheres to the following rules. Otherwise, it is not considered acceptable.

- Rule 1: The number of beads in the sequence is between 5 and 10, inclusive.
- Rule 2: The sequence contains at least one blue bead (represented by "B") and at least one white bead (represented by "W").
- Rule 3: The sequence starts and ends with the same color.

You will create three boolean variables to help determine whether the sequence is acceptable, and then print a message indicating the result. The following table illustrates the intended behavior for different values of beadSequence.

Value of beadSequence	Meets Rule 1	Meets Rule 2	Meets Rule 3	Printed Output
"OBWO"	false	true	true	"Not acceptable"
"BWBGOGWBB"	true	true	true	"Acceptable"
"ROYGBIVW"	true	true	false	"Not acceptable"
"BOBOBOBO"	true	false	false	"Not acceptable"
"WBWRWBWRWBWR"	false	true	false	"Not acceptable"
"BWWWBBWWWB"	true	true	true	"Acceptable"

12. Write a code segment that creates a boolean variable named correctColors and assigns it true if beadSequence meets rule 2 and assigns it false otherwise.

13. Write a code segment that creates a boolean variable named lengthGood and assigns it true if beadSequence meets rule 1 and assigns it false otherwise.

14. Write a code segment that creates a boolean variable named startEndGood and assigns it true if beadSequence meets

rule 3 and assigns it false otherwise.

15. Write a code segment that prints "Acceptable" if beadSequence adheres to all three rules and prints "Not acceptable" otherwise. You must use the variables lengthGood, correctColors, and startEndGood to receive full credit, regardless of what you wrote for parts A, B, and C.

---

16. **Directions: SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.**

Notes:

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

4. This question involves analyzing strings. You will write the `countRepeat` method of the following `StringAnalyzer` class.

```
public class StringAnalyzer
{
 /** Returns a count of how many times smallStr occurs in largeStr, as
  *   described in part (a)
  *   Precondition: largeStr is not null; smallStr is not null.
  *                 The length of smallStr is less than the length of largeStr.
  */
 public static int countRepeat(String largeStr, String smallStr)
 {  /* to be implemented in part (a) */  }

 // There may be variables and methods that are not shown.
}
```

(a) The `countRepeat` method is used to count and return the number of times the string parameter `smallStr` appears in the string parameter `largeStr`. In the case of overlapping occurrences of `smallStr`, only the first occurrence is included in the overall count, as shown in the second and third calls to `countRepeat` in the following table of examples.

Call to `countRepeat`	**Return Value**
`StringAnalyzer.countRepeat("BAAB", "AA")`	1
`StringAnalyzer.countRepeat("AAAAA", "AA")`	2
`StringAnalyzer.countRepeat("AABABABAA", "ABA")`	2
`StringAnalyzer.countRepeat("ABBAABB", "ABA")`	0

Complete method `countRepeat`.

FINAL Unit1-Unit2 FRQ - Topic order

```
/** Returns a count of how many times smallStr occurs in largeStr,
 *   as described in part (a)
 *   Precondition: largeStr is not null; smallStr is not null.
 *                 The length of smallStr is less than the length of largeStr.
 */
public static int countRepeat(String largeStr, String smallStr)
```

(b) A programmer would like to modify the `StringAnalyzer` class to limit the length of the large string that can be passed as input to `countRepeat`. The limit could vary between calls to `countRepeat`. A call to `countRepeat` with a value of `largeStr` that is longer than the maximum length would return `-1`. The programmer would like to implement this change without making any changes to the signature of the `countRepeat` method or overloading `countRepeat`.

Write a description of how you would change the `StringAnalyzer` class in order to support this modification. **Do not write the program code for this change.**

Make sure to include the following in your response.

- Identify any new or modified variables or methods.
- Describe, for each new or revised variable or method, how it would change or be implemented, including visibility and type.

This question involves processing an email address to create three usernames.

The email address is stored in a String variable named email, which consists of a first name, followed by ".", followed by a last name, followed by "@", followed by a domain name. Assume that the email address is stored in this format and the last name has at least three characters.

The usernames will be formatted as follows.

- Username 1 will consist of the first name, followed by "_", followed by the last name.
- Username 2 will consist of the first initial of the first name, followed by "_", followed by the last name.
- Username 3 will consist of a randomly-selected character from the first name , followed by "_", followed by a sequence of three consecutive characters selected from a random position in the last name.

Example 1

Suppose email is equal to "joseph.martin@school.org".

- Username 1 will be "joseph_martin".
- Username 2 will be "j_martin".
- Possible values of Username 3 include "s_art" and "p_tin".

Example 2

Suppose email is equal to "virginia.farmingham@school.net".

- Username 1 will be "virginia_farmingham".
- Username 2 will be "v_farmingham".

FINAL Unit1-Unit2 FRQ - Topic order

- Possible values of Username 3 include "g_ham" and "i_arm".

17. Write a statement that uses the String method indexOf to find the index of the first "." in email and assign it to a variable. Write another statement that uses the String method indexOf to find the index of "@" and assign it to a variable.

18. Write a statement that uses the String method substring to extract the first name from email and assign it to a variable. Write another statement that uses the String method substring to extract the last name from email and assign it to a variable. Then use the results of those statements to create and assign the correct value to a String variable for username 1.

19. Write a statement that uses the String method substring to extract the first initial of the first name from email and assign it to a variable. Use the results of this statement and the statements you wrote in part B to create and assign the correct value to a String variable for username 2.

20. Write a code segment that uses the random numbers that you generated in part D to create and assign the correct value to a String variable for username 3.

---

21. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves the `AppointmentBook` class, which provides methods for students to schedule appointments with their teacher. Appointments can be scheduled during one of eight class periods during the school day, numbered 1 through 8. A requested appointment has a duration, which is the number of minutes the appointment will last. The 60 minutes within a period are numbered 0 through 59. In order for an appointment to be scheduled, the teacher must have a block of consecutive, available minutes that contains at least the requested number of minutes in a requested period. Scheduled appointments must start and end within the same period.

The `AppointmentBook` class contains two helper methods, `isMinuteFree` and `reserveBlock`. You will write two additional methods of the `AppointmentBook` class.

```
public class AppointmentBook
{
/**
 * Returns true if minute in period is available for an appointment and
returns
 * false otherwise
 * Preconditions: 1 <= period <= 8; 0 <= minute <= 59
 */
private boolean isMinuteFree(int period, int minute)
```

FINAL Unit1-Unit2 FRQ - Topic order

```
{ /* implementation not shown */ }


/**
* Marks the block of minutes that starts at startMinute in period and
* is duration minutes long as reserved for an appointment
* Preconditions: 1 <= period <= 8; 0 <= startMinute <= 59;
* 1 <= duration <= 60
*/
private void reserveBlock(int period, int startMinute, int duration)
{ /* implementation not shown */ }


/**
* Searches for the first block of duration free minutes during period, as
described in
* part (a). Returns the first minute in the block if such a block is found
or returns -1 if no
* such block is found.
* Preconditions: 1 <= period <= 8; 1 <= duration <= 60
*/
public int findFreeBlock(int period, int duration)
{ /* to be implemented in part (a) */ }


/**
* Searches periods from startPeriod to endPeriod, inclusive, for a block
* of duration free minutes, as described in part (b). If such a block is
found,
* calls reserveBlock to reserve the block of minutes and returns true;
otherwise
* returns false.
* Preconditions: 1 <= startPeriod <= endPeriod <= 8; 1 <= duration <= 60
*/
public boolean makeAppointment(int startPeriod, int endPeriod,
                                                    int duration)
{ /* to be implemented in part (b) */ }

// There may be instance variables, constructors, and methods that are not
shown.
}
```

Write the `findFreeBlock` method, which searches `period` for the first block of free minutes that is `duration` minutes long. If such a block is found, `findFreeBlock` returns the first minute in the block.

FINAL Unit1-Unit2 FRQ - Topic order

Otherwise, `findFreeBlock` returns `-1`. The `findFreeBlock` method uses the helper method `isMinuteFree`, which returns `true` if a particular minute is available to be included in a new appointment and returns `false` if the minute is unavailable.

Consider the following list of unavailable and available minutes in period 2.

Minutes in Period 2	Available?
0 – 9 (10 minutes)	No
10 – 14 (5 minutes)	Yes
15 – 29 (15 minutes)	No
30 – 44 (15 minutes)	Yes
45 – 49 (5 minutes)	No
50 – 59 (10 minutes)	Yes

The method call `findFreeBlock(2, 15)` would return `30` to indicate that a 15-minute block starting with minute `30` is available. No steps should be taken as a result of the call to `findFreeBlock` to mark those 15 minutes as unavailable.

The method call `findFreeBlock(2, 9)` would also return `30`. Whenever there are multiple blocks that satisfy the requirement, the earliest starting minute is returned.

The method call `findFreeBlock(2, 20)` would return `-1`, since no 20-minute block of available minutes exists in period 2.

Complete method `findFreeBlock`. You must use `isMinuteFree` appropriately in order to receive full credit.

```
/**
* Searches for the first block of duration free minutes during period, as
described in
* part (a). Returns the first minute in the block if such a block is found or
returns -1 if no
* such block is found.
* Preconditions: 1 <= period <= 8; 1 <= duration <= 60
*/
public int findFreeBlock(int period, int duration)
```

Write the `makeAppointment` method, which searches the periods from `startPeriod` to `endPeriod`, inclusive, for the earliest block of `duration` available minutes in the lowest-numbered period. If such a block is found, the `makeAppointment` method calls the helper method `reserveBlock` to mark the minutes in the block as unavailable and returns `true`. If no such block is found, the `makeAppointment` method returns `false`.

Consider the following list of unavailable and available minutes in periods 2, 3, and 4 and three successive calls to `makeAppointment`.

Period	Minutes	Available?
2	0 – 24 (25 minutes)	No
2	25 – 29 (5 minutes)	Yes
2	30 – 59 (30 minutes)	No
3	0 – 14 (15 minutes)	Yes
3	15 – 40 (26 minutes)	No
3	41 – 59 (19 minutes)	Yes
4	0 – 4 (5 minutes)	No
4	5 – 29 (25 minutes)	Yes

FINAL Unit1-Unit2 FRQ - Topic order

Period	Minutes	Available?
4	30 – 43 (14 minutes)	No
4	44 – 59 (16 minutes)	Yes

The method call `makeAppointment(2, 4, 22)` returns `true` and results in the minutes 5 through 26, inclusive, in period 4 being marked as unavailable.

The method call `makeAppointment(3, 4, 3)` returns `true` and results in the minutes 0 through 2, inclusive, in period 3 being marked as unavailable.

The method call `makeAppointment(2, 4, 30)` returns `false`, since there is no block of 30 available minutes in periods 2, 3, or 4.

The following shows the updated list of unavailable and available minutes in periods 2, 3, and 4 after the three example method calls are complete.

Period	Minutes	Available?
2	0 – 24 (25 minutes)	No
2	25 – 29 (5 minutes)	Yes
2	30 – 59 (30 minutes)	No
3	0 – 2 (3 minutes)	No
3	3 – 14 (12 minutes)	Yes
3	15 – 40 (26 minutes)	No
3	41 – 59 (19 minutes)	Yes
4	0 – 26 (27 minutes)	No
4	27 – 29 (3 minutes)	Yes
4	30 – 43 (14 minutes)	No
4	44 – 59 (16 minutes)	Yes

Complete method `makeAppointment`. Assume that `findFreeBlock` works as intended, regardless of what you wrote in part (a). You must use `findFreeBlock` and `reserveBlock` appropriately in order to receive full credit.

```
/**
* Searches periods from startPeriod to endPeriod, inclusive, for a block
* of duration free minutes, as described in part (b). If such a block is found,
* calls reserveBlock to reserve the block of minutes and returns true; otherwise
* returns false.
* Preconditions: 1 <= startPeriod <= endPeriod <= 8; 1 <= duration <= 60
*/
public boolean makeAppointment(int startPeriod, int endPeriod,
                               int duration)
```